

Construction Replay Transcript

HL7, EDI & FHIR Data Integration for Analytics & Reporting · Wednesday, September 2, 2026 · running time 8:27

Shooting script: timecode, where to click in eiConsole, then the spoken line.

0:00 – 0:09

[Official open cards]

This is PilotFish and the eiConsole, and this is a demo for HL7, EDI & FHIR Data Integration for Analytics & Reporting.

0:09 – 0:18

[Click “Aggregation”]

This is a demonstration of the PilotFish eiConsole, a graphical integrated development environment for the rapid configuration of any Healthcare Interface.

0:18 – 0:52

[Click “HIE”]

Here are several example routes that represent real-world uses of the software. Data acquisition and normalization of Health Information Exchange data. Receipt and transformation of laboratory orders, status and results. On-premise medical equipment integration, such as smart cards in a hospital that must be integrated with pharmacy, EMR and billing systems. Acquisition of clinical and administrative data for reporting and analytics. Integration of HL7-based hospital systems with cloud-based solution providers that may be using technology such as JSON, XML and web services.

0:52 – 0:59

[Double-click package “Aggregation for Analytics”]

Now let's take a more in-depth look at this multiple Source route to reporting and analytics example.

0:59 – 1:09

[Double-click route “HL7 - EDI - FHIR”]

Double-clicking a route takes us to the main route grid, which graphically depicts the flow of data between Source and Target systems.

1:09 – 1:24

[Click “RoutingSource”]

[Route grid — click the Source System column]

Here we see data coming in as HL7 over LLP. We have some claims-based EDI files being dropped in an FTP folder. We also have some FHIR JSON coming in through a RESTful web service.

1:24 – 1:34

[Hold — Source System]

In each case, we'll extract some basic patient information and use it to populate a relational database and execute a simple REST-based JSON service.

1:34 – 1:48

[Hold — Source System]

To do this, the user configures the eiConsole Interface Assembly Line, a consistent set of stages flexible enough to connect any system to any other, regardless of data format or connectivity protocol.

1:48 – 1:56

[Hold — Source System]

Moving from left to right, we start with the Source system, which maintains descriptive information about the sending system.

1:56 – 2:09

[Route grid — click the Listener column]

Next is the Listener – the entry point of the interface. To configure a Listener, we select the appropriate listener type from the drop-down and fill out the related property panels.

2:09 – 2:24

[Click “HL7 Source System”]

The eiConsole supports real-time, periodic, scheduled and triggered listeners. Some commonly implemented listeners include HL7 over LLP, FTP including SFTP and FTPS, SOAP and RESTful based web services, and messaging middleware such as JMS, MSMQ, and RabbitMQ.

2:24 – 2:41

[Hold — Listener]

After the Listeners, configured processors offer a library of widgets for manipulation and validation before data transformation. Common examples include decryption, decompression or validation. Processors, as with other stages in the assembly line, are extensible through an open Java-based API.

2:41 – 2:53

[Route grid — click the Source Transform column]

After pre-processing, we move on to the Source Transformation. Here the data is converted from the inbound format into a canonical XML representation through a two-step process.

2:53 – 3:08

[Click “HL7 to XML”]

[Click Format Profile]

[Click Transformation Module]

First, parsing to convert data into XML. Transformation modules convert a variety of non-XML formats, including CSVs, delimited and fixed-width files, EDI ANSI X12 and other HIPAA formats, HL7, JSON, Microsoft Excel and others.

3:08 – 3:24

[Hold — Source Transform]

Note that the HL7 transformation module has been specifically designed to be lenient. Given all the various versions and non-standard flavors of HL7 messages, this ensures that virtually anything that even looks like HL7 can be parsed for future processing.

3:24 – 3:33

[Click the To XML tab]

The second step of transformation is logical mapping, which is always an XML to XML transformation represented as XSLT.

3:33 – 3:43

[Route grid — click the Source Transform column]

[Click Edit]

This is the Data Mapper that allows a technical business analyst to drag and drop to build transformations between any two data formats.

3:43 – 3:50

[Click the Mapping tab]

The 3-panel mapping paradigm is unique in its ability to represent even the most complex mappings graphically.

3:50 – 4:04

[Click Open source format]

To use the mapper, the Source and Target data formats are built right in through the format builder dialogue. Various types of metadata such as the HL7 data dictionary can be directly imported.

4:04 – 4:12

[Click Open target format]

Sample files can also be used to provide an accurate picture of what a system may actually produce or consume.

4:12 – 4:30

[Click PID]

[Click patient]

[Click PID]

Note also that when using HL7, Friendly Names can be applied to the elements to make it easier to determine what fields. Here HL7 was loaded as the Source, and a simple XML sample file was used as the Target.

4:30 – 4:36

[Click lastName]

Dragging and dropping these fields together in the center panel creates a one-to-one map.

4:36 – 4:56

[Click firstName]

[Click dob]

The palette of functions above the mapping allows for conditional logic, looping logic, data manipulation and much more. All of this can be accomplished without custom or proprietary scripting; however, the mapping is directly based on XSLT, a standard transformation language with a strong developer community.

4:56 – 5:14

[Click the XSLT tab]

[Click xsl:template]

[Click the Mapping tab]

[Click Return to Console]

[Click Don't Save]

[Click "RoutingSource"]

[Route grid — click the Routing column]

Returning to the main grid, the next step is Routing. Here XPath or other routing rules can be used to inspect the data and direct the inbound message toward the appropriate Target system or systems.

5:14 – 5:20

[Hold — Routing]

Proactive error notification via the transaction monitor metaphor is also configured at this point.

5:20 – 5:28

[Route grid — click the Target Transform column]

We then move to the Target side of the interface. Each Target system has an accompanying Transformation and Transport.

5:28 – 5:49

[Click "XML to DB"]

[Click Format Profile]

[Click the From XML tab]

The Target Transform is the mirror image of the Source, transforming the canonical XML format to the format required by the Target system. Note that on the Target side, the Transformation Module executes second, converting the output of any logical mapping into the physical format required by the Target.

5:49 – 6:03

[Route grid — click the Target Transform column]

[Click "XML to DB"]

[Click Format Profile]

[Click Edit]

[Click Edit]

[Click the Mapping tab]

[Click Open source format]

[Click Open target format]

[Click patient]

[Click Insert]

[Click Insert]

On this Target we map the canonical onto SQLXML and indicate an insert into a table called patient.

6:03 – 6:15

[Click the XSLT tab]

[Click the Mapping tab]

[Click Return to Console]

[Click Don't Save]

[Route grid — click the Transport column]

Finally, we move on to the Transport stage, and in this case, we're updating a relational database and also calling a RESTful web service.

6:15 – 6:25

[Click "Populate Database"]

However, as is true with a Listener, a wide variety of transmission mechanisms are supported. Both synchronous and asynchronous processing is also possible.

6:25 – 6:42

[Hold — Transport]

Pre and post processes are also available in the Transport step for encrypting, compressing or otherwise manipulating the data prior to transmission. In addition, post processes could be used for cleanup tasks and executing subsequent steps in a complex data flow.

6:42 – 6:50

[Hold — Testing Mode]

Once an interface has been configured in the eiConsole, it can then be switched into testing mode.

6:50 – 6:58

[Click Testing Mode]

In Test mode, we can test a route from end-to-end or choose a subset of stages we wish to test.

6:58 – 7:14

[Click Execute Test]

When we start in the middle of the process, we're asked to supply a sample file manually. As the file is processed, successful stages are marked with a green check and failures are indicated with a red X.

7:14 – 7:37

[Hold — Testing Mode]

You can then look at how the data appeared at each stage in our process. Here we supplied an HL7 ADT message for input. The message was converted into XML with the Friendly Names by the HL7 transformation module. Finally, the Source mapping extracted key pieces of patient demographic information into a simple XML canonical.

7:37 – 7:57

[Hold — Testing Mode]

The data was routed to both defined Targets. The canonical was mapped first onto SQLXML. Here we indicate an insert into a table called patient and second, we simply rendered some JSON. We then will update the database and post it to that RESTful web service.

7:57 – 8:15

[Hold — Testing Mode]

Now that an interface has been successfully tested from end-to-end, it's ready for deployment to the eiPlatform runtime environment. Deployment can be handled via drag and drop using the server view, also the PilotFish web-based eiDashboard, or through your preferred file-based change management process.

8:15 – 8:27

[Hold — Testing Mode]

Once deployed, the eiPlatform provides a myriad of reporting, alerting and monitoring utilities. The eiPlatform is used for Production Management Interfaces. And that's all there is to it!